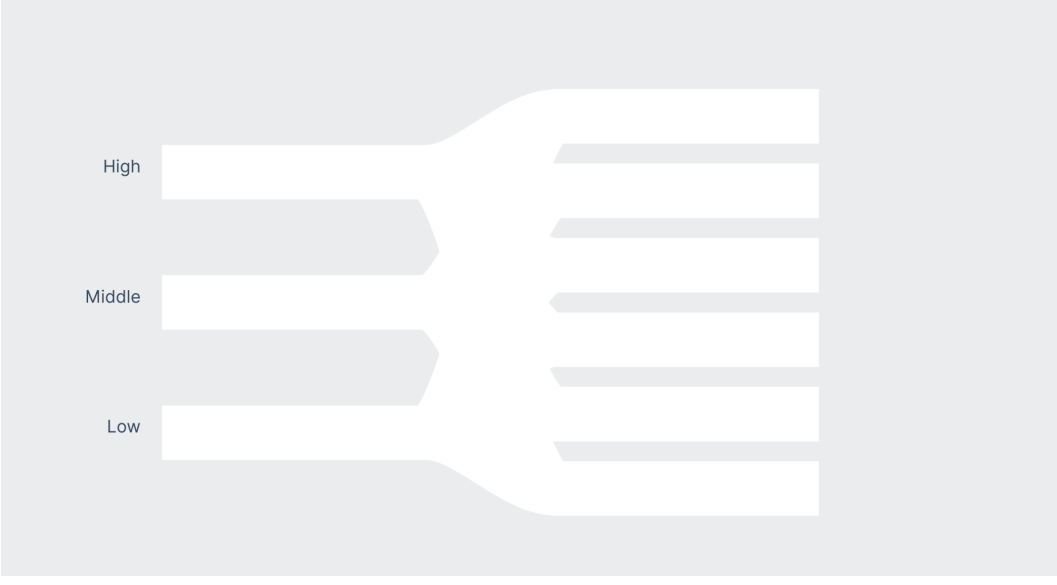
code/12-animated-sankey/completed/chart.js

```
160   const startingLabels = startingLabelsGroup.selectAll(".start-label")
161     .data(sesIds)
162     .enter().append("text")
163       .attr("class", "label start-label")
164       .attr("y", (d, i) => startYScale(i))
165       .text((d, i) => sentenceCase(sesNames[i]))
```

Let's align them to the right of our group, using the `text-anchor` CSS property in our `styles.css` file. We'll also vertically center them with our paths using the `dominant-baseline` CSS property.

```
.start-label {
    text-anchor: end;
    dominant-baseline: middle;
}
```



Starting labels

These labels aren't very descriptive, though. A viewer isn't likely to know what **High** means without any context. Let's add a title above our start labels.

There's one issue though: the phrase **Socioeconomic status** is pretty long and won't easily fit where we want it to. SVG `<text>` elements don't wrap to multiple lines the way HTML text does. We'll need to create two `<text>` elements, positioned one on top of the other.

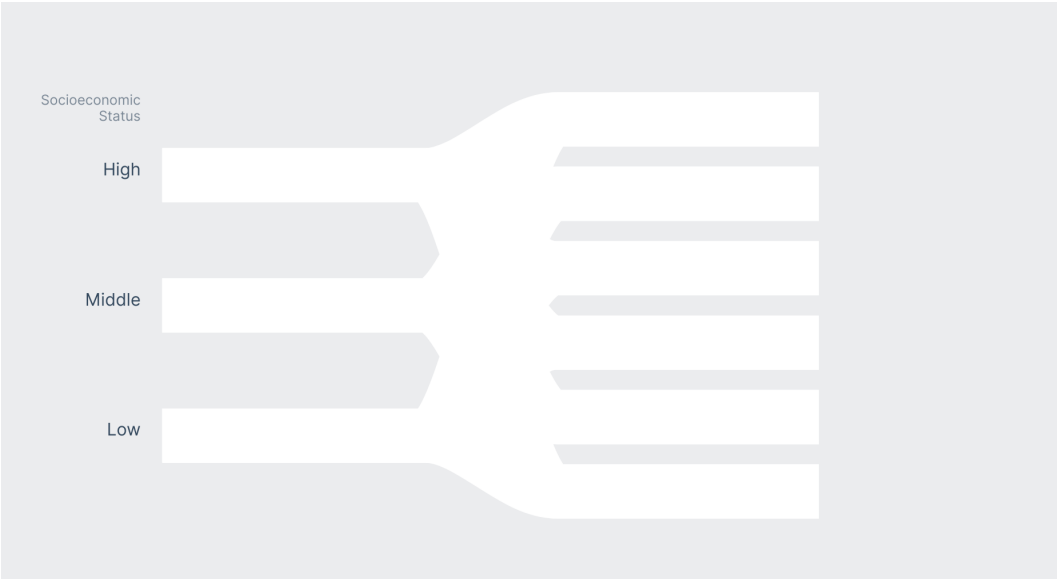
code/12-animated-sankey/completed/chart.js

```
167   const startLabel = startingLabelsGroup.append("text")
168       .attr("class", "start-title")
169       .attr("y", startYScale(sesIds[sesIds.length - 1]) - 65)
170       .text("Socioeconomic")
171   const startLabelLineTwo = startingLabelsGroup.append("text")
172       .attr("class", "start-title")
173       .attr("y", startYScale(sesIds[sesIds.length - 1]) - 50)
174       .text("Status")
```

Let's add a few styles to align our title with our start titles, and dim the opacity to make it visually distinct.

```
.start-title {
    text-anchor: end;
    font-size: 0.8em;
    opacity: 0.6;
}
```

Great! Now viewers can see which each starting position means.



Starting title and labels

End labels

For our ending labels, we'll want to label the paths, but also show how many "people" of each socioeconomic status and gender ended up in each path. Let's start with the labels, but leave room underneath to show those numbers.

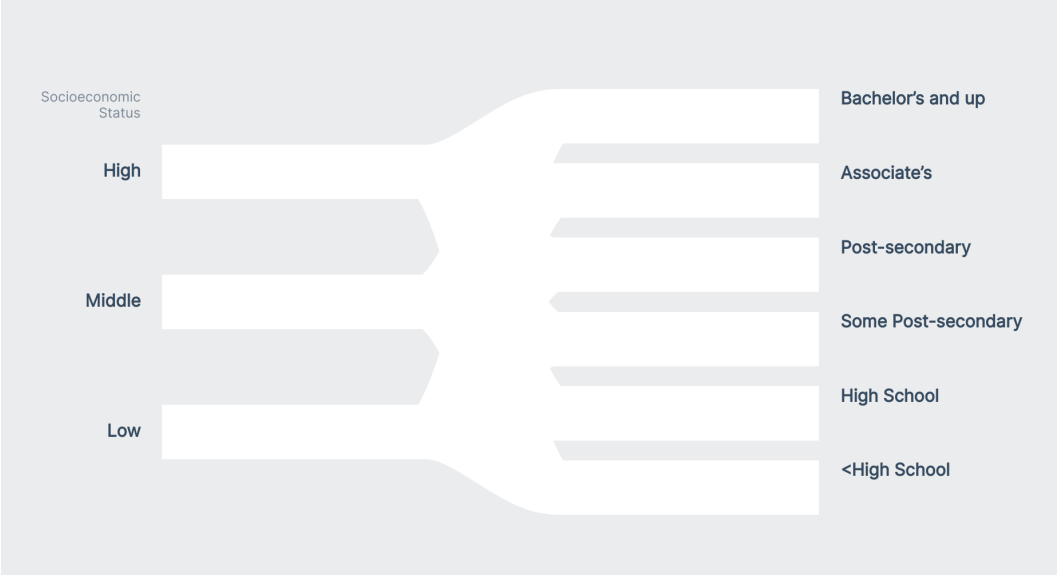
code/12-animated-sankey/completed/chart.js

```
185   const endingLabelsGroup = bounds.append("g")
186       .style("transform", `translateX(${
187         dimensions.boundedWidth + 20
188       }px)`)
189
190   const endingLabels = endingLabelsGroup.selectAll(".end-label")
191     .data(educationNames)
192     .enter().append("text")
193       .attr("class", "label end-label")
194       .attr("y", (d, i) => endYScale(i) - 15)
195       .text(d => d)
```

Notice that we used the `"label"` class name on both our start and end labels – let's give these a little more visual weight since they're more important than other annotations like the title of our starting labels and the values we'll add to our ending labels.

```
.label {
    font-weight: 600;
    dominant-baseline: middle;
}
```

Great! That should be weighty enough.

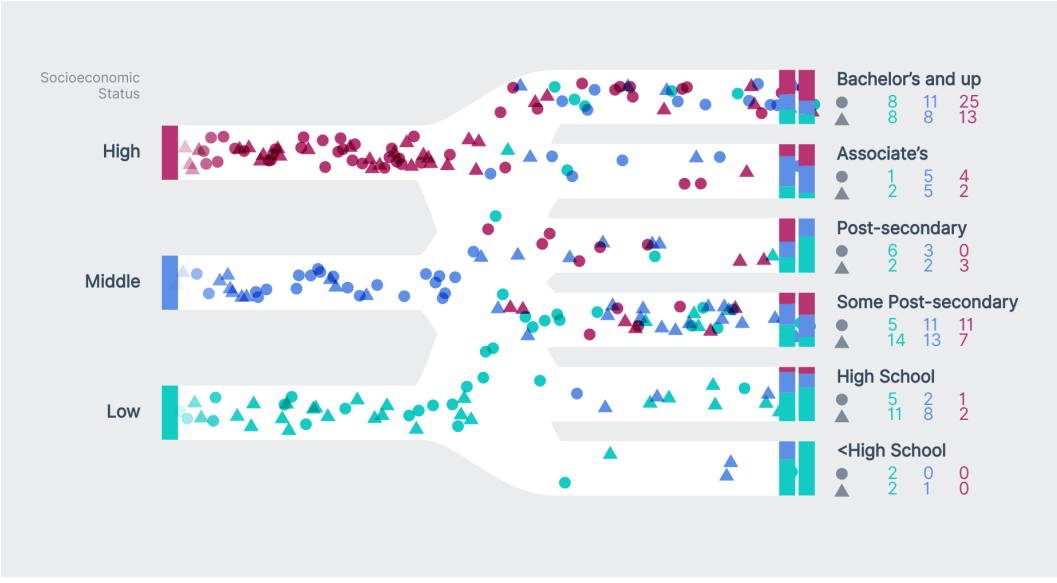


Ending labels

Drawing the ending markers

We want to represent males with a circle and females with a triangle. These shapes are simple enough to prevent over-complicating the final diagram, but visually distinct enough to allow viewers to parse them quickly.

We'll show the amount of people who made it into each ending position by showing males in a row on the top and females in a row on the bottom. We'll be using color to signify socioeconomic status.



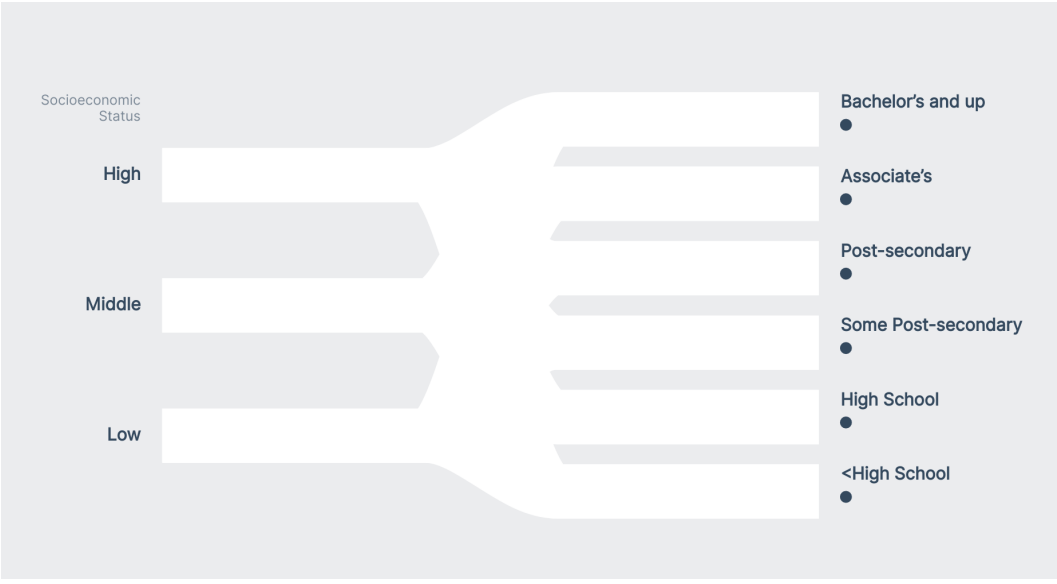
Ending values example

Let's start by adding a `<circle>` underneath each ending position label, we'll add the values when we start animating our "people". If we refer to our **SVG Cheatsheet** (in **Appendix C**), we'll see that `<circle>`s are positioned with a `cx` and `cy` attribute. The `c` declares that these coordinates refer to the *center of the circle*, instead of the *top right*.

code/12-animated-sankey/completed/chart.js

```
197   const maleMarkers = endingLabelsGroup.selectAll(".male-marker")
198     .data(educationIds)
199     .enter().append("circle")
200       .attr("class", "ending-marker male-marker")
201       .attr("r", 5.5)
202       .attr("cx", 5)
203       .attr("cy", d => endYScale(d) + 5)
```

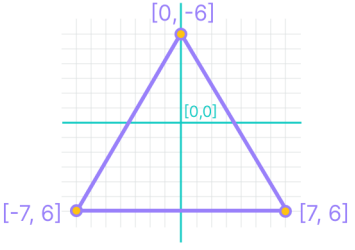
Perfect!



Ending labels with circles

Next, we'll want to draw triangles to signify our second row of female counts. However, there is no `<triangle>` SVG element – we'll need to build one ourselves.

Since a triangle is a fairly basic shape, let's define the `points` of a `<polygon>` element. We'll make our triangle almost equilateral, but the height will be a *tiny* bit shorter than the width, to give it a friendlier appearance.



Triangle diagram

Let's start with the *bottom, left* point and work around our triangle in a clockwise fashion.

code/12-animated-sankey/completed/chart.js

```
  const trianglePoints = [
    "-7,  6",
    " 0, -6",
    " 7,  6",
  ].join(" ")
```

The `.join()` method will combine our separate coordinates into one string:

```
"-7, 6 0, -6 7, 6"
```

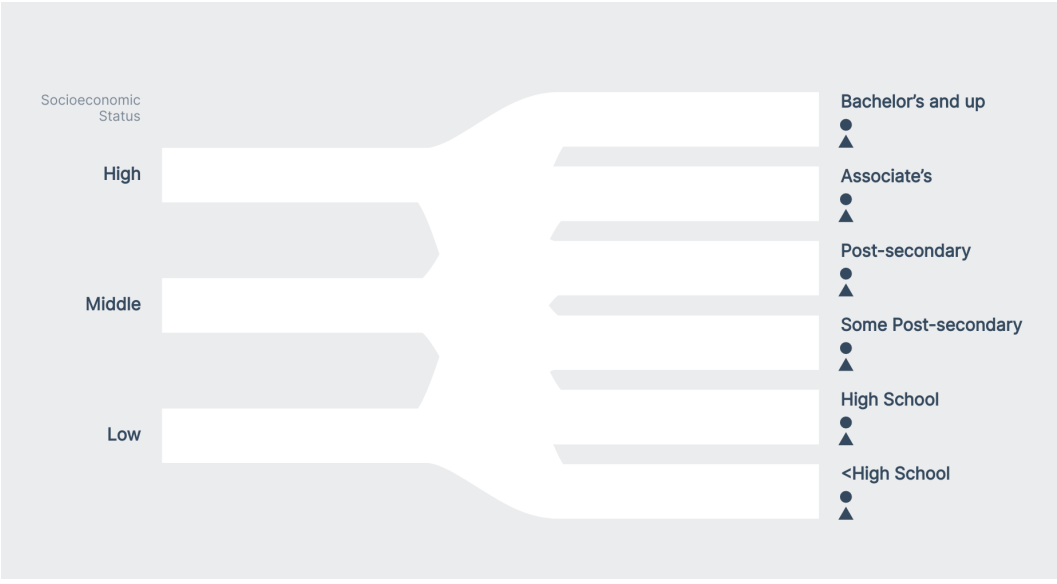
We could have written the string directly, but creating an array and `.join()`ing the points is easier to parse and modify, if needed.

Let's draw our polygon markers – our **SVG Cheatsheet** tells us that there are no *position* attributes for `<polygon>`s, so we'll use `transform` instead.

code/12-animated-sankey/completed/chart.js

```
  const femaleMarkers = endingLabelsGroup.selectAll(".female-marker")
    .data(educationIds)
    .enter().append("polygon")
      .attr("class", "ending-marker female-marker")
      .attr("points", trianglePoints)
      .attr("transform", d => `translate(5, ${endYScale(d) + 20})`)
```

Looking good!

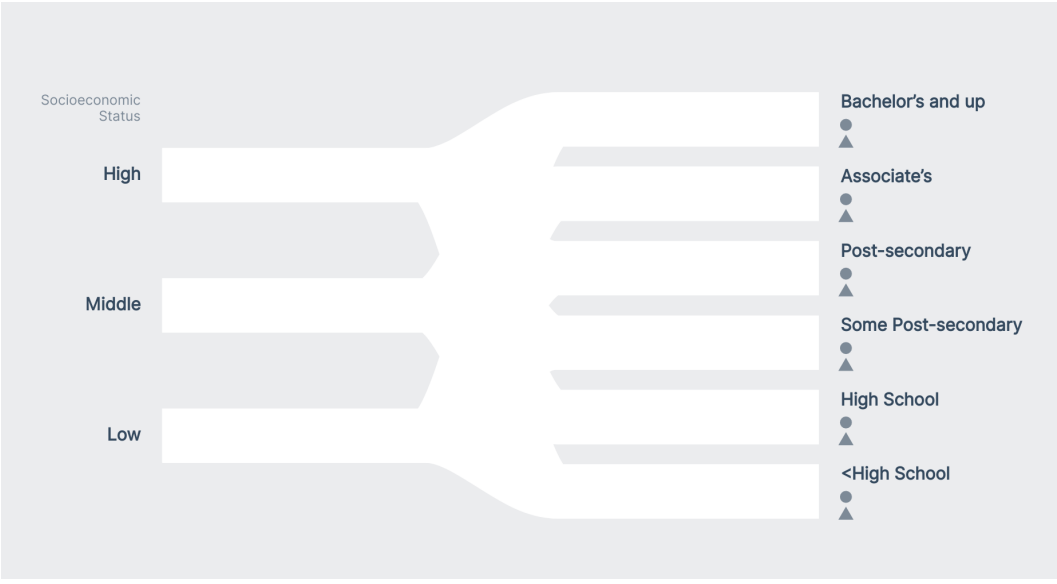


Ending markers

Those markers distract a bit from our ending labels – let's dim them in our `styles.css` file.

```
.ending-marker {
    opacity: 0.6;
}
```

Perfect!



Ending markers, faded

We're all set with labels for now – viewers will now be able to tell what each path signifies.

Drawing people

We're all ready to dig in and start drawing our "people". In out **Set up interactions** step, let's begin by initializing two variables:

1. our `people` list that will hold all of our simulated people
2. our `<g>` element that will hold all of our people markers

```
let people = []
const markersGroup = bounds.append("g")
    .attr("class", "markers-group")
```

Next, we need to make a function called `updateMarkers()` that will draw our people. We'll use `d3.timer()`[80] to update the position of our people.

[80] https://github.com/d3/d3-timer#timer

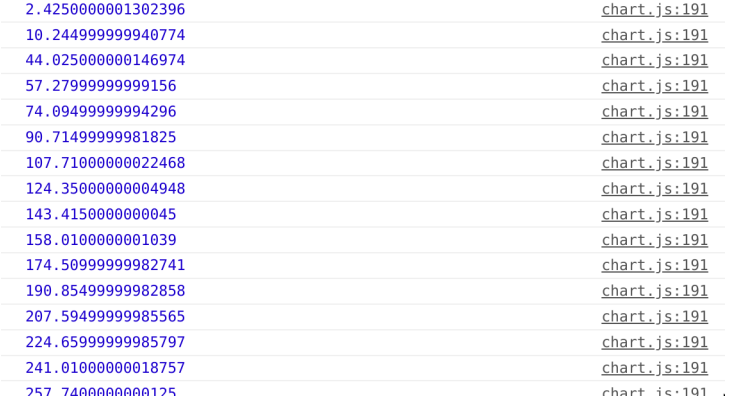
`d3.timer()`'s first parameter is a callback function that it will call until the timer is stopped (which we can do the timer's `.stop()` method). This callback function will have access to one parameter: how many milliseconds have elapsed since the timer started.

Let's create our `updateMarkers()` function and pass it to `d3.timer()` to call. To check what `d3.timer()` is handing `updateMarkers()`, let's log out the first parameter.

```
function updateMarkers(elapsed) {
    console.log(elapsed)
}
d3.timer(updateMarkers)
```

> `d3.timer()`[a] has other performance goodies built-in, like using `requestAnimationFrame()`[b] for smooth animations, and pausing when the window isn't visible.
>
> [a] https://github.com/d3/d3-timer
> [b] https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame

If we look in our Dev Tools **Console**, we'll see that we're rapidly logging out the milliseconds elapsed.

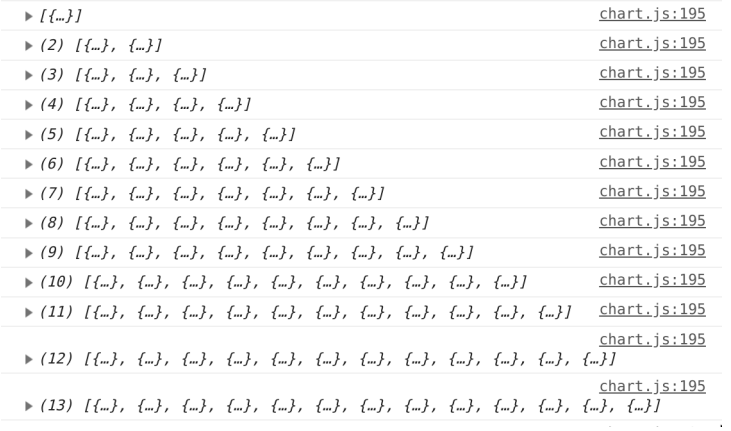


Timestamps in the console

Perfect! Instead of logging the `elapsed`, let's add a new person to our `people` array every time `updateMarkers()` runs.

```
function updateMarkers(elapsed) {
      people = [
        ...people,
        generatePerson(),
      ]
      console.log(people)
}
```

Wonderful – we can see that our array contains a new person on every iteration.



People array in the console

Starting with females, let's draw a `<circle>` for every person in our array with a `sex` of `0`. We can isolate these people by `.filter()`ing our `people` array.

```
const females = markersGroup.selectAll(".marker-circle")
  .data(people.filter(d => sexAccessor(d) == 0))

females.enter().append("circle")
  .attr("class", "marker marker-circle")
  .attr("r", 5.5)
```

Since we're not positioning our `<circle>`s, they show up in the *top, left* of our **bounds**.

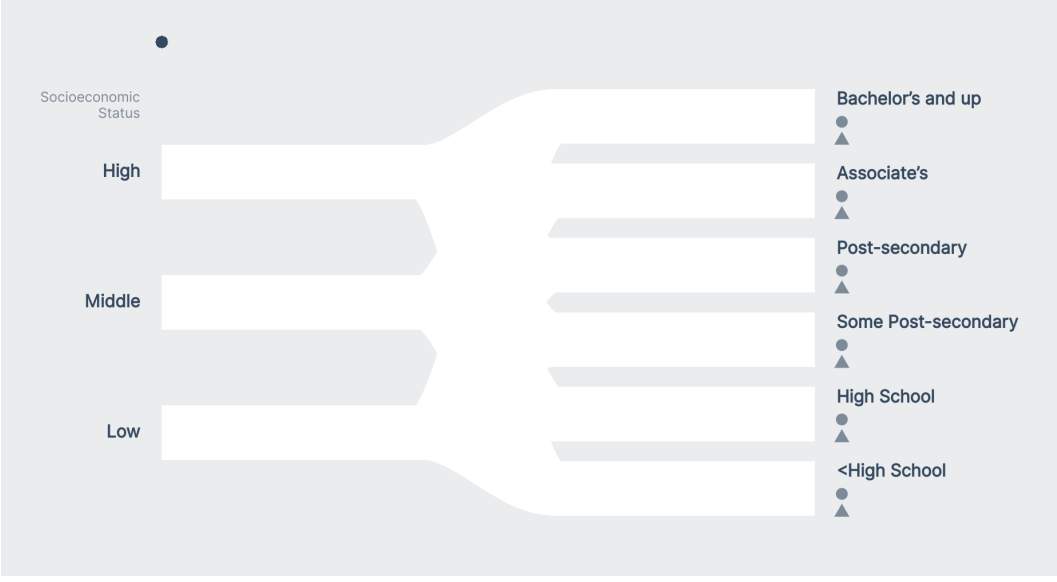


Chart with circles in the top, left

Let's draw our male markers, then update all of our markers' positions at once.

```
const males = markersGroup.selectAll(".marker-triangle")
 .data(people.filter(d => sexAccessor(d) == 1))
males.enter().append("polygon")
 .attr("class", "marker marker-triangle")
 .attr("points", trianglePoints)
```

Positioning our people

Let's start our markers 10 pixels to the left of our starting line, and vertically position them with their `ses` property's path.

```
const markers = d3.selectAll(".marker")

markers.style("transform", d => {
  const x = -10
  const y = startYScale(sesAccessor(d))
  return `translate(${ x }px, ${ y }px)`
})
```

Great, that's exactly where we want our markers to start – at the beginning of our paths.

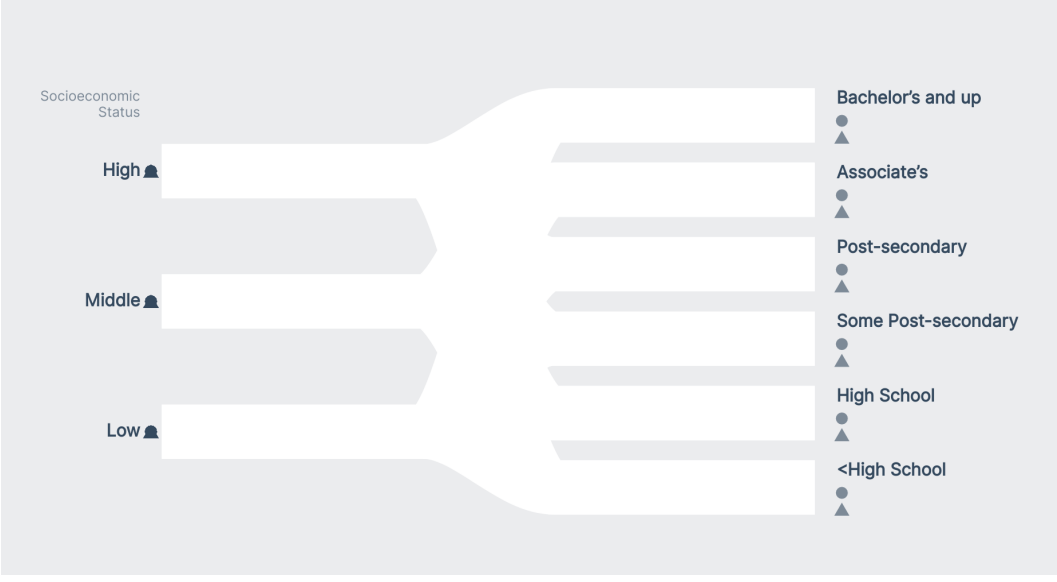


Chart with markers at the start

We want our chart to be animated, though! Let's add the `elapsed` value to our markers' x-position.

```
markers.style("transform", d => {
   const x = elapsed
   // ...
```

There we go! Our markers skitter off to the right. But we don't want to draw all of our people on top of each other – we want each person to start at the left when they are created.

Let's pass our `elapsed` milliseconds to each person as they are created.

```
people = [
   ...people,
   generatePerson(elapsed),
]
```

Then we can update our `generatePerson()` function to record *when that person was created.*

```
function generatePerson(elapsed) {
 // ...

 return {
     sex,
     ses,
     education,
     startTime: elapsed,
 }
}
```

And now, when we draw each person, we'll subtract their `startTime` from their x-position.

```
markers.style("transform", d => {
    const x = elapsed - d.startTime
    // ...
```

Nice! Now we can see a steady stream of markers making their way across the page.

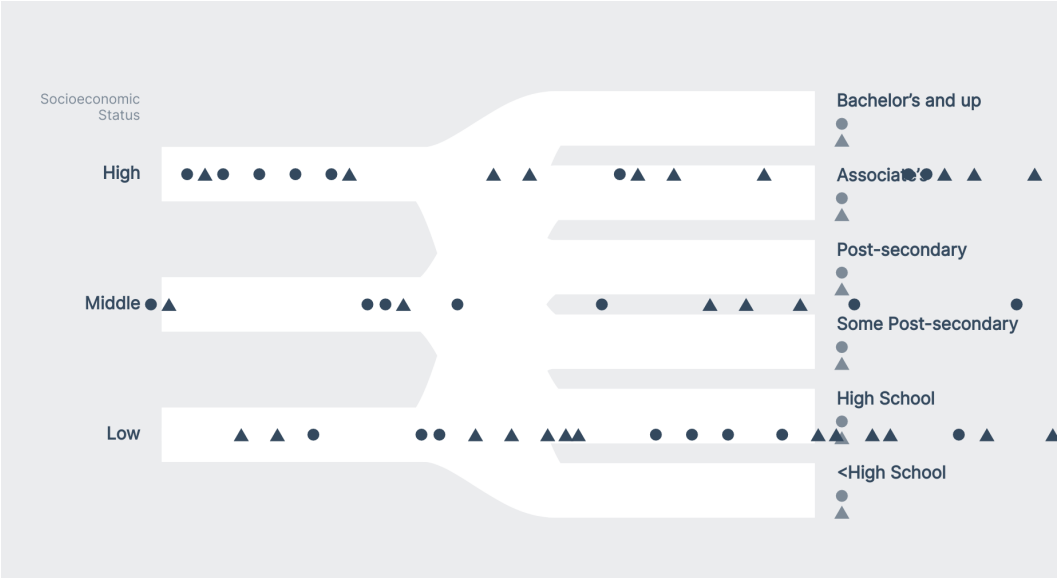
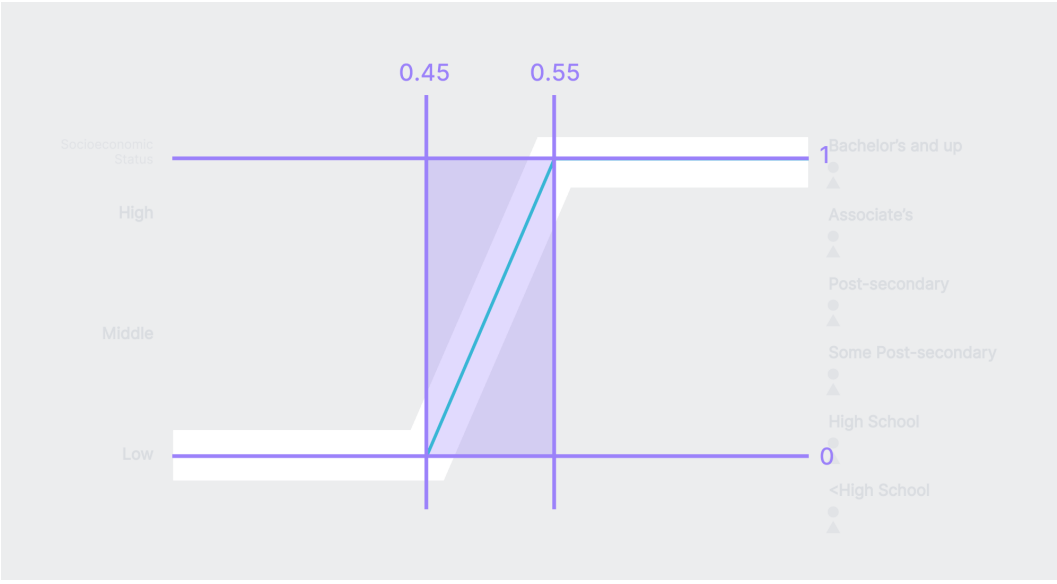


Chart with staggered markers

Updating our peoples' y-positions

Next, let's update our markers' y-positions so that follow one of our paths and end up in the correct education bucket on the right. We'll need a scale that converts a person's progress along their path into a y-position.

If we had one starting y-position and one ending y-position, we would be able to make a scale from A to B. Unfortunately, we have *many* paths that we need to interpolate between. Instead, let's create a scale that converts an **x-position progress** (`0` to `1`) into a **y-position percent**.



yTransitionProgressScale diagram

We'll clamp our scale so that the lowest value it returns is `0` and the highest is `1`.

> This code goes at the bottom of our **Create scales** step.

code/12-animated-sankey/completed/chart.js

```
121   const yTransitionProgressScale = d3.scaleLinear()
122     .domain([0.45, 0.55]) // x progress
123     .range([0, 1])        // y progress
124     .clamp(true)
```

Let's move back down to our `updateMarkers()` function. We'll need an easy way to get a person's x-progress along their path. Let's create an `xProgressAccessor` that assumes that **a person takes 5 seconds (5000 milliseconds) to cross the chart**.

```
function updateMarkers(elapsed) {
 const xProgressAccessor = d => (elapsed - d.startTime) / 5000
 // ...
```

Now let's update our function where we set our markers' positions – we'll find each person's x-progress and pass it to our `xScale` to get their correct x position. We'll then find their starting and ending y positions, and update their y-position based on their progress along the route.

```
markers.style("transform", d => {
 const x = xScale(xProgressAccessor(d))
 const yStart = startYScale(sesAccessor(d))
 const yEnd = endYScale(educationAccessor(d))
 const yChange = yEnd - yStart
 const yProgress = yTransitionProgressScale(xProgressAccessor(d))
 const y = yStart
     + (yChange * yProgress)
 return `translate(${ x }px, ${ y }px)`
})
```

Much better! Now our markers stay in their starting y-position until they reach the middle of the chart, then they gradually transition to their ending y-position.

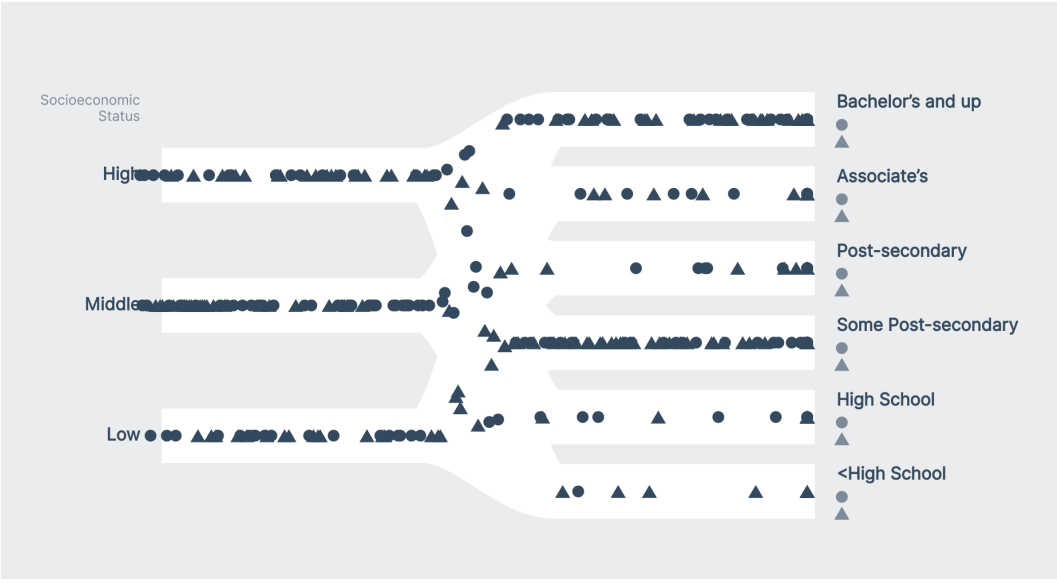


Chart with markers following their paths

Adding jitter

How can we make it easier to follow individual dots? Because our dots are sticking to the middle of their paths, they run into each other and are hard to distinguish.

Let's give each person a y-jitter – we'll want to do this when they are created, since we want the jitter to be consistent across their journey. We can use the `getRandomNumberInRange()` utility function that is defined at the bottom of our `chart.js` file.

Let's also jitter each person's `startTime`, so their x positions aren't so regular. This will give our visualization a bit more of a natural feel.

```
function generatePerson(elapsed) {
  // ...
 return {
     sex,
     ses,
     education,
     startTime: elapsed + getRandomNumberInRange(-0.1, 0.1),
     yJitter: getRandomNumberInRange(-15, 15),
 }
}
```

Bingo! Now each marker has more room, making its shape easier to parse.

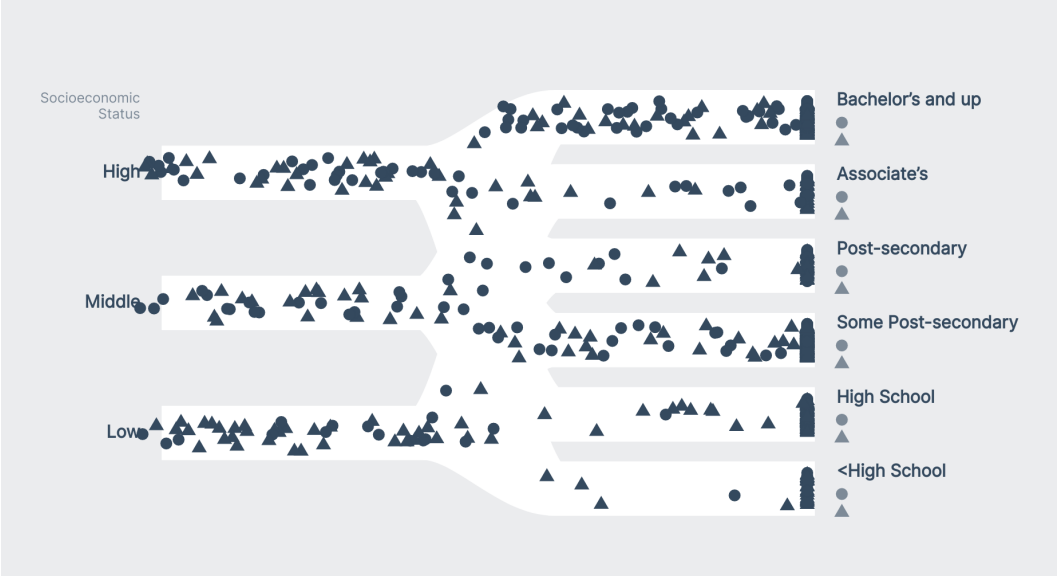


Chart with jittered markers

Hiding people off-screen

Once our simulation has been running for a while, we'll have a lot of elements on the screen at once, which can get expensive. Let's clean up and markers that have finished their journey by adding another filter rule when we create our female markers:

```
const females = markersGroup.selectAll(".marker-circle")
  .data(people.filter(d => (
    xProgressAccessor(d) < 1
    && sexAccessor(d) == 0
  )))
  // ...
```

and our male markers:

```
const males = markersGroup.selectAll(".marker-triangle")
  .data(people.filter(d => (
    xProgressAccessor(d) < 1
    && sexAccessor(d) == 1
  )))
  // ...
```

We'll want to remove any markers that have completed their journey. While we're in our "markers drawing" code, let's also initialize each marker with an `opacity` of `0`, so we can fade them in at the start.

```
females.enter().append("circle")
    .attr("class", "marker marker-circle")
    .attr("r", 5.5)
    .style("opacity", 0)
females.exit().remove()


males.enter().append("polygon")
    .attr("class", "marker marker-triangle")
    .attr("points", trianglePoints)
    .style("opacity", 0)
males.exit().remove()
```

And once our dots are at least 10 pixels to the right of their starting position, let's fade them in.

```
markers.style("transform", d => {
      // ...
    })
  .transition().duration(100)
    .style("opacity", d => xScale(xProgressAccessor(d)) < 10 ? 0 : 1)
```

Alright! Now our chart is only showing as many markers as is necessary.

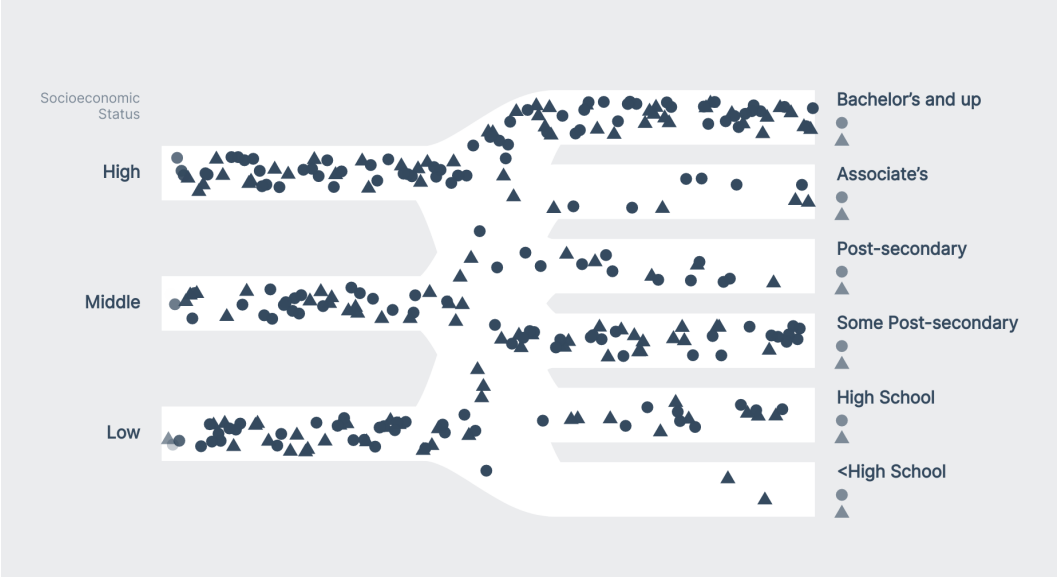


Chart with disappearing markers

Adding color

Once our markers make it to the right side, their **sex** is still clear (based on their shape), but it's not clear what **socioeconomic status** they started in. Let's add a color scheme that to help distinguish our markers.

Creating a color scale

Let's create a color scale at the end of our **Create scales** step – thankfully this will be the last scale we need for this visualization! We can use a **linear scale** that

interpolates between two colors – by setting the **domain** to an array of our `sesIds`, we'll get three unique, equally-spaced colors.

code/12-animated-sankey/completed/chart.js

```
126   const colorScale = d3.scaleLinear()
127     .domain(d3.extent(sesIds))
128     .range(["#12CBC4", "#B53471"])
129     .interpolate(d3.interpolateHcl)
```

> We're using `.interpolate()` to specify that we want to use the **hcl color space**. If you remember from **Chapter 7**, **hcl** manipulates colors with human perception in mind. Play around with different color spaces by changing the `.interpolation()` setting to get a sense of how the color spaces differ.

Adding a color key

There are various ways to signify to the viewer what **socioeconomic status** each color stands for. We *could* add a legend, but a more elegant solution is to add colored bars at the start of each path. This way, the legend is built into the chart, and we don't need to repeat ourselves.

Since we'll be adding these color bars to the start *and* end of each path, let's add an `endsBarWidth` constant to our `dimensions` object:

```
let dimensions = {
   // ...
   pathHeight: 50,
   endsBarWidth: 15,
}
```

Now we can draw our bars in our **Draw peripherals** step, right after we label our starting paths.

code/12-animated-sankey/completed/chart.js

```
176   const startingBars = startingLabelsGroup.selectAll(".start-bar")
177     .data(sesIds)
178     .enter().append("rect")
179       .attr("x", 20)
180       .attr("y", d => startYScale(d) - (dimensions.pathHeight/ 2))
181       .attr("width", dimensions.endsBarWidth)
182       .attr("height", dimensions.pathHeight)
183       .attr("fill", colorScale)
```

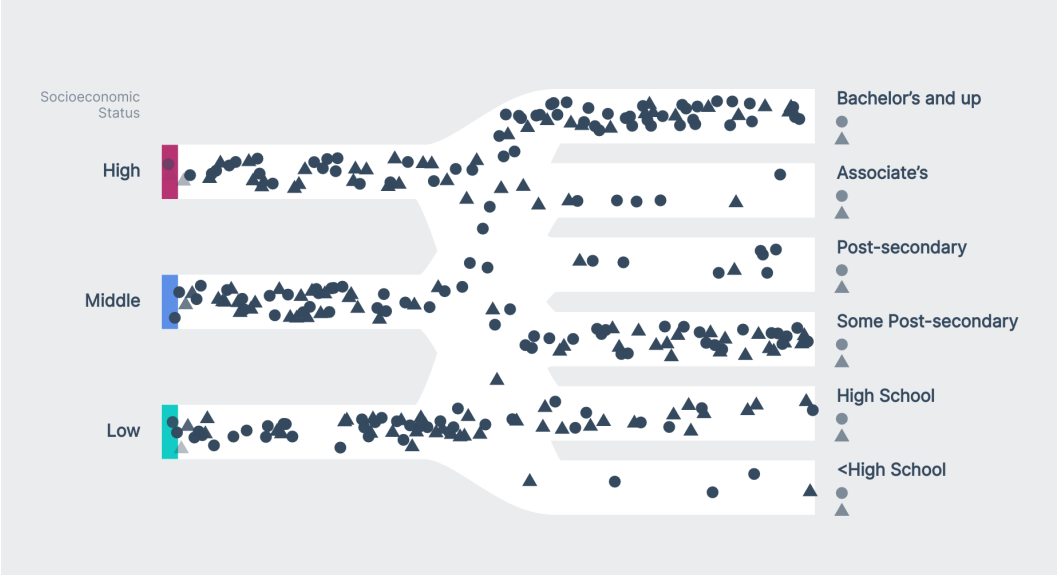


Chart with starting bars

Coloring our markers

Now that we have our "legend" in place, let's color our markers. We can update their `fill` just before we transition their `opacity` (at the end of our `updateMarkers()` function).

```
markers.style("transform", d => {
      // ...
    })
    .attr("fill", d => colorScale(sesAccessor(d)))
  .transition().duration(100)
    .style("opacity", d => xScale(xProgressAccessor(d)) < 10 ? 0 : 1)
```

Awesome! Now we can tell where each person started, and trends are easier to notice. For example, most of the markers traveling along in the bottom, right are green, and the ending paths have more and more pink markers as we move up the **education** buckets.

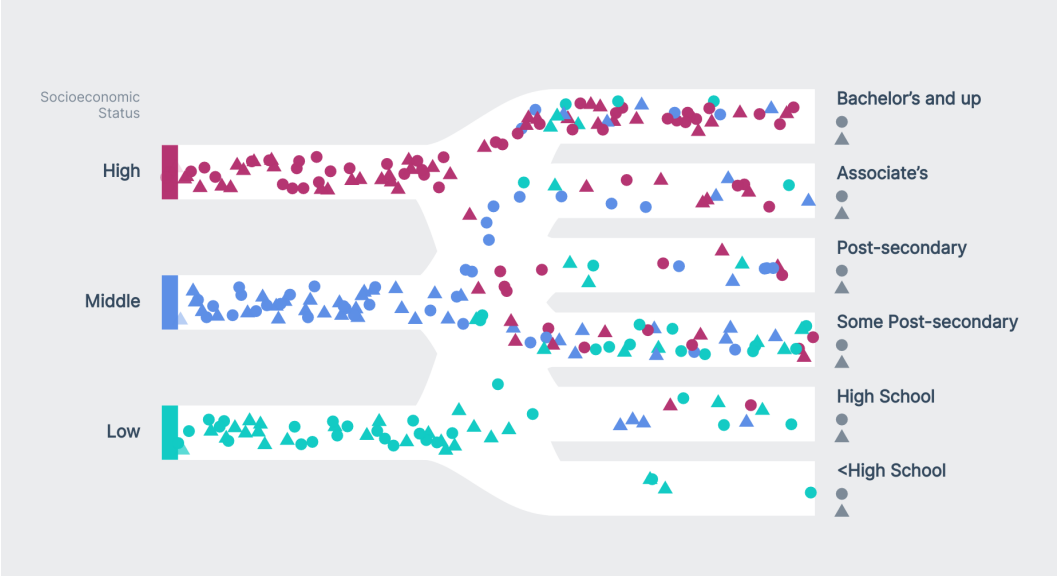


Chart with colored markers

Adding a filter to our markers

Let's make one more tweak to help our viewers parse the markers' shapes. Since our markers are *solid*, they cover each other completely when they are overlapped. Let's add a `blend-mode` to darken any overlaps, which will help complete each individual marker's shape.

```
.marker {
    mix-blend-mode: multiply;
}
```

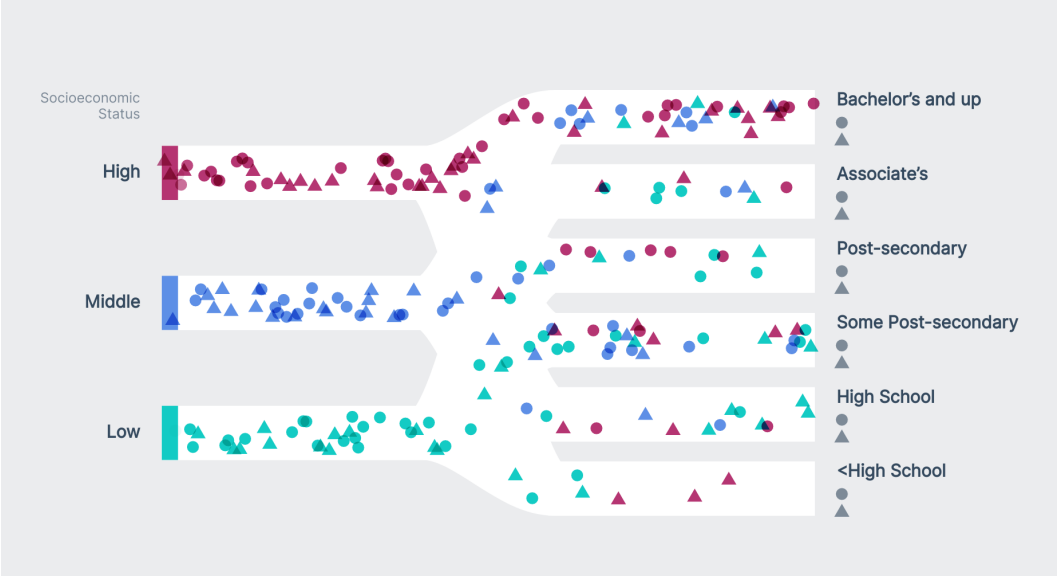


Chart with colored markers, with a filter

Even though the overall visualization doesn't change very much, if we zoom in we can see that each shape is more distinct.

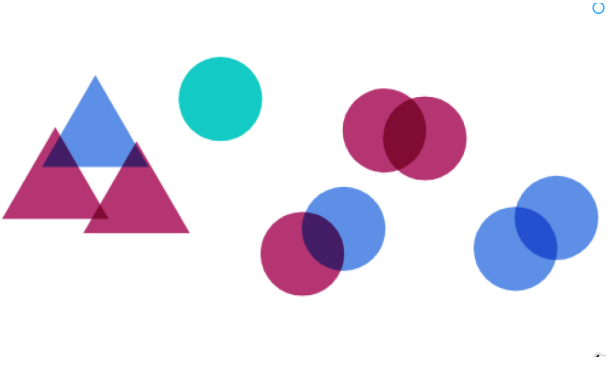


Chart with colored markers, with a filter (zoomed-in)

Giving our people ids

Notice how our markers overlap our start bars, once our markers reach the right side? This is because our existing people are being recycled, and already have an `opacity` of `0`.

Let's give each person a unique `id` when we create them. Each time we call `generatePerson()`, we'll increment a `currentPersonId` variable that we'll use as an `id`. This way, each person's `id` will be distinct.

```
let currentPersonId = 0
function generatePerson(elapsed) {
  currentPersonId++

  // ...
  return {
    id: currentPersonId,
    // ...
  }
}
```

When we create our female and male markers in our `updateMarkers()` function, we can tell d3 how to distinguish people from one another. D3 selection objects' `.data()` method[81] takes a second parameter: a **key accessor**. This **key accessor** defaults to the element's **index** – this explains why our markers were being recycled: items in our filtered array are removed once they reach the end of their journey, and new elements end up in the same position in the filtered list.

Instead, let's set our **key accessor** functions to return a person's `id`, which will guarantee that they aren't recycled.

[81] https://github.com/d3/d3-selection#selection_data

```
const females = markersGroup.selectAll(".marker-circle")
  .data(people.filter(d => (
    xProgressAccessor(d) < 1
    && sexAccessor(d) == 0
  )), d => d.id)


const males = markersGroup.selectAll(".marker-triangle")
  .data(people.filter(d => (
    xProgressAccessor(d) < 1
    && sexAccessor(d) == 0
  )), d => d.id)
```

Great! Now that our markers aren't being recycled, they will always start with an `opacity` of `0`.

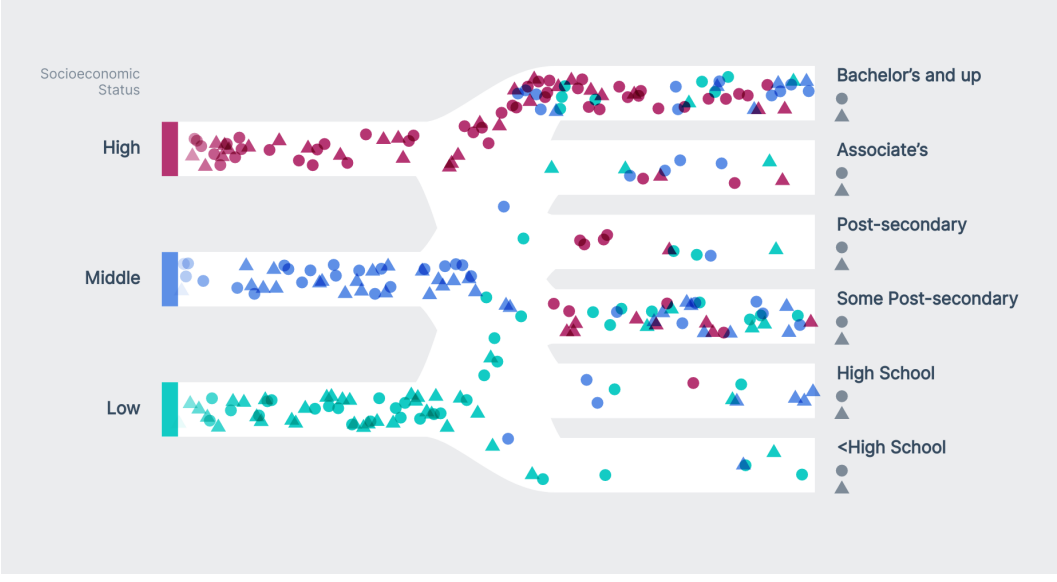


Chart that doesn't recycle people

Showing ending numbers

We can get some information about trends by looking at the flow of colors and shapes, but let's show some concrete numbers. Let's mimic the colored bars on our start positions, using stacked bars to show the proportion of people who reached that **education** bucket, based on their starting **socioeconomic status**.

We could draw one bar per ending position, but we want to also show how the proportions differ based on the person's **sex**. Let's draw two bars: one for **females** and one for **males**.

First, we'll add a new variable to our `dimension` object: `endingBarPadding`. This will determine how far apart our **female** and **male** bars are.

```
let dimensions = {
    // ...
  endingBarPadding: 3,
}
```

Let's create a `<g>` to contain our ending bars. Since we run `updateMarkers()` multiple times, we'll want to create this group outside of it, to prevent from creating multiple groups.

```
const endingBarGroup = bounds.append("g")
    .attr("transform", `translate(${dimensions.boundedWidth}, 0)`)

function updateMarkers(elapsed) {
    // ...
```

Next, let's draw our ending bars. At the end of our `updateMarkers()` function, let's create an array of people who have finished their journey and fit inside of that **education** bucket.

```
function updateMarkers(elapsed) {
    // ...
   const endingGroups = educationIds.map((endId, i) => (
     people.filter(d => (
       xProgressAccessor(d) >= 1
       && educationAccessor(d) == endId
     ))
   ))
```

We'll want to draw one bar per path: each permutation of **sex**, **socioeconomic status**, and **educational attainment**. Let's create a flattened array, with one object per permutation that contain the **starting** and **ending** positions, the count, the percent above, and the total count in the bar.

```
const endingPercentages = d3.merge(
  endingGroups.map((peopleWithSameEnding, endingId) => (
    d3.merge(
      sexIds.map(sexId => (
        sesIds.map(sesId => {
          const peopleInBar = peopleWithSameEnding.filter(d => (
            sexAccessor(d) == sexId
          ))
          const countInBar = peopleInBar.length
          const peopleInBarWithSameStart = peopleInBar.filter(d => (
            sesAccessor(d) == sesId
          ))
          const count = peopleInBarWithSameStart.length
          const numberOfPeopleAbove = peopleInBar.filter(d => (
            sesAccessor(d) > sesId
          )).length
          return {
            endingId,
            sesId,
            sexId,
            count,
            countInBar,
```

```
            percentAbove: numberOfPeopleAbove
              / (peopleInBar.length || 1),
            percent: count / (countInBar || 1),
          }
        })
      ))
    )
  ))
)
```

Now that we have an array for each of our ending bars, we can create one `<rect>` per bar.

```
endingBarGroup.selectAll(".ending-bar")
  .data(endingPercentages)
  .join("rect")
    .attr("class", "ending-bar")
    .attr("x", d => -dimensions.endsBarWidth * (d.sexId + 1)
      - (d.sexId * dimensions.endingBarPadding)
    )
    .attr("width", dimensions.endsBarWidth)
    .attr("y", d => endYScale(d.endingId)
      - dimensions.pathHeight / 2
      + dimensions.pathHeight * d.percentAbove
    )
    .attr("height", d => d.countInBar
      ? dimensions.pathHeight * d.percent
      : dimensions.pathHeight
    )
    .attr("fill", d => d.countInBar
      ? colorScale(d.sesId)
      : "#dadadd"
    )
```

Now once our markers finish their journey, we'll see their colors populate the bars on the right. After our simulation has been running for a while, our stacked bars should start to level out, approximating the percentages in our dataset.

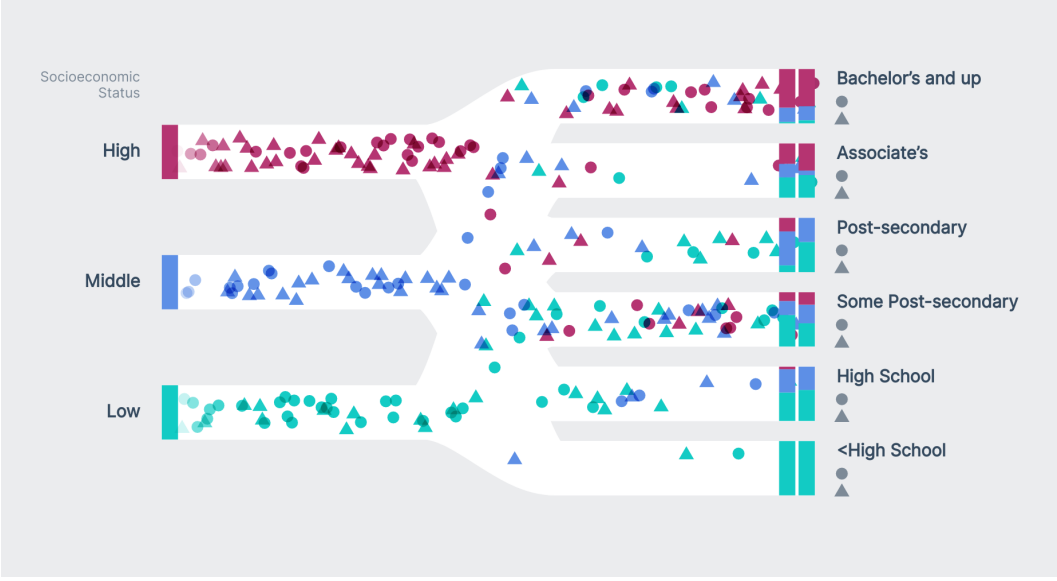


Chart with ending bars

The changes in our bars' heights can be jerky at first – let's smooth those transitions with a `transition` CSS property.

```
.ending-bar {
    transition: all 0.3s ease-out;
}
```

Updating the ending values

What if our users want the exact values of these bars? Let's show the exact counts underneath the labels for each of our path endings, next to our **female** and **male** markers.

```
endingLabelsGroup.selectAll(".ending-value")
  .data(endingPercentages)
  .join("text")
    .attr("class", "ending-value")
    .attr("x", d => (d.sesId) * 33
      + 47
    )
    .attr("y", d => endYScale(d.endingId)
      - dimensions.pathHeight / 2
      + 14 * d.sexId
      + 35
    )
    .attr("fill", d => d.countInBar
      ? colorScale(d.sesId)
      : "#dadadd"
    )
    .text(d => d.count)
```

Great! Now we can see the exact numbers updating as more of our markers finish.

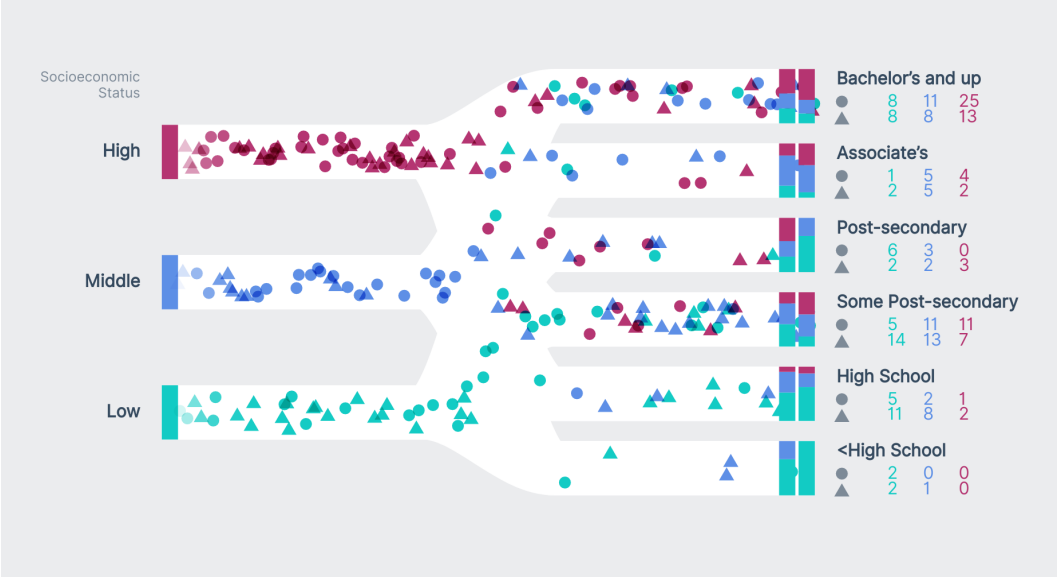


Chart with ending values

Let's make a few tweaks to our numbers' styles: we'll decrease their size, right-align them, and fix their width so they horizontally align with each other.

```
.ending-value {
    font-size: 0.7em;
    text-anchor: end;
    font-weight: 600;
    font-feature-settings: 'tnum' 1;
}
```

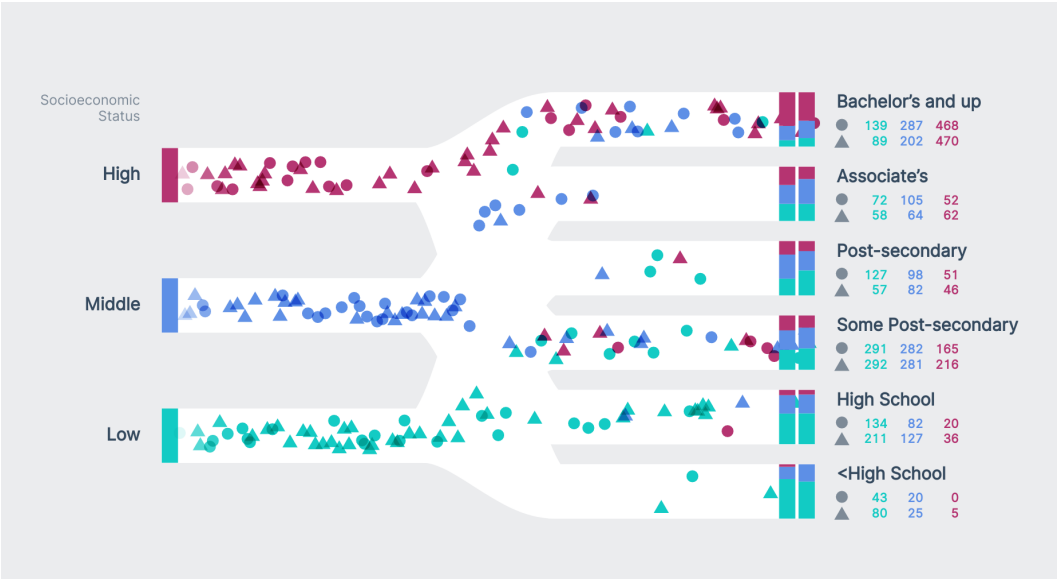


Chart with ending values, styled

Label our ending bars

We're almost there! We have one more tweak to make to our chart.

We never explain what each marker shape means, or what both of our ending bars stand for. Let's fix both of these issues at once, by adding a label above these bars.

At the end of our **Draw peripherals** step, we'll create a <g> for our legend.

```
const legendGroup = bounds.append("g")
    .attr("class", "legend")
    .attr("transform", `translate(${dimensions.boundedWidth}, 5)`)
```

Next, we'll create another <g>, this time specifically to label the left ending bars. We'll move this group to the left to sit right above the center of these bars.

```
const femaleLegend = legendGroup.append("g")
    .attr("transform", `translate(${
      - dimensions.endsBarWidth * 1.5
      + dimensions.endingBarPadding
      + 1
    }, 0)`)
```

In this group, we'll draw a female marker, `<text>` to label our marker, and a `<line>` that connects our marker to the stacked bars.

```
femaleLegend.append("polygon")
    .attr("points", trianglePoints)
    .attr("transform", "translate(-7, 0)")
femaleLegend.append("text")
    .attr("class", "legend-text-left")
    .text("Female")
    .attr("x", -20)
femaleLegend.append("line")
    .attr("class", "legend-line")
    .attr("x1", -dimensions.endsBarWidth / 2 + 1)
    .attr("x2", -dimensions.endsBarWidth / 2 + 1)
    .attr("y1", 12)
    .attr("y2", 37)
```

Let's add a few styles in our `styles.css` file to fix our label's text alignment and give our `<line>` a stroke.

```
.legend {
    font-size: 0.8em;
    opacity: 0.6;
    dominant-baseline: middle;
}

.legend-text-left {
    text-anchor: end;
}

.legend-line {
    stroke: grey;
    stroke-width: 1px;
}
```

Great! Now our viewers can tell what each shape means, and what each stacked bar stands for.

Chart with legend, female

Lastly, let's label our **male** stacked bars.

```
const maleLegend = legendGroup.append("g")
    .attr("transform", `translate(${
      - dimensions.endsBarWidth / 2
      - 4
    }, 0)`)
maleLegend.append("circle")
    .attr("r", 5.5)
    .attr("transform", "translate(5, 0)")
maleLegend.append("text")
    .attr("class", "legend-text-right")
    .text("Male")
    .attr("x", 15)
    .attr("y", 0)
maleLegend.append("line")
    .attr("class", "legend-line")
    .attr("x1", dimensions.endsBarWidth / 2 - 3)
    .attr("x2", dimensions.endsBarWidth / 2 - 3)
    .attr("y1", 12)
    .attr("y2", 37)
```

And voila! Our visualization is finished.

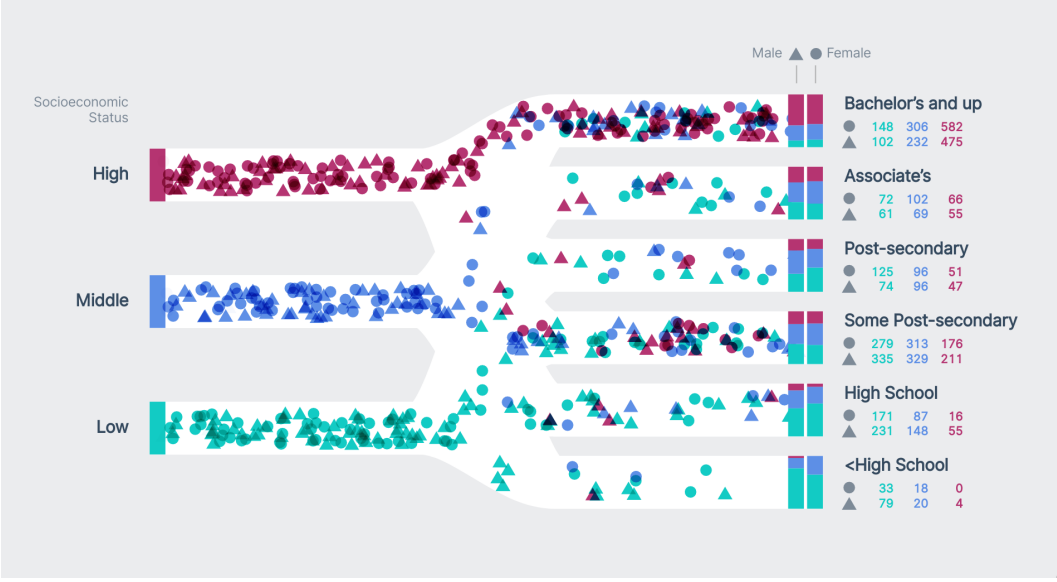


Chart with legend

Additional steps

If you were interested in building on this chart for a production visualization, you would want to add a threshold for a maximum number of people. Left running for a while, the list of `people` would grow and grow, until it was too large and crashed the browser window. Check out the completed code for an example of how to implement a threshold.

Wrapping up

Great job making it through this visualization, it's the most complex one by far! Show it off by sharing an image or a gif with friends!

We learned a lot along the way – for example, how to simulate data and how to create an animation loop. I hope you see how simulating a dataset engages a reader in a way that a static visualization doesn't.

Using D3 With React

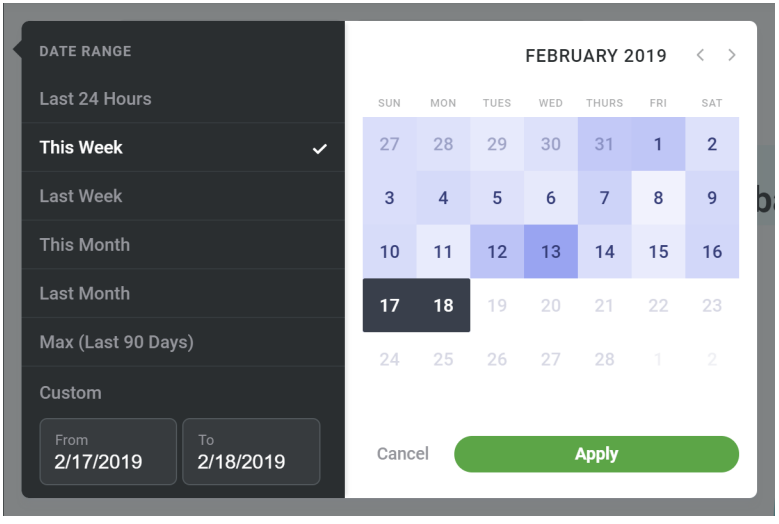
We know how to make individual charts, but you might ask: what's the best way to draw a chart within my current JavaScript framework? D3 can be viewed as a utility library, but **it's also used to manipulate the DOM**, which means that there is a fair bit of overlap in functionality between a JavaScript framework like React and d3 — let's talk about the best way to handle that overlap.

First off, we should decide **when** to use d3. Should we use d3 to render a whole page?

Let's split up d3's functionality by concern:

1. DOM manipulation (like jQuery)
2. Data manipulation (manipulation, interpolation, basic stats)

With the library compartmentalized in this way, you might come up with unorthodox ways to utilize d3. For example, I recently used it to create a calendar date picker.



Date picker

This date picker doesn't look like a chart, but d3 came in handy in a few ways.

- **d3-date** helps with splitting up a date range into weeks
- **d3-date** helps with calculating date range defaults — for example, if a user selects **Last Week**, the date picker will use d3.timeWeek.floor() and d3.timeWeek.ceil() to find the start and end of the week.
- **d3-scale** helps with creating a color scale to show data values between each day. This helps users know which days to select based on the data (in this case, online attention to a specific topic).
- **d3-time-format** helps to display each day's day of the month and the input values on the bottom left.
- **d3-selection** helps create mouse events to select days on hover when a user has selected their start date and will select their end date.

A d3 novice might not think to utilize d3 in this way, but once you've read this book you will be familiar enough to take full advantage of the d3 library.

React.js

React.js is a framework for building declarative, module-based user interfaces. It helps you split your interface code into components, which each have a `render` function that describes the resulting DOM structure. One of React's greatest strengths is its diffing algorithm, which ensures minimal DOM updates, which are relatively expensive.

If you're unfamiliar with React, spend some time running through an introduction like this one[82]. Our walkthrough will assume a basic understanding of the core concepts since we want to focus on the **d3 *and* React** bits.

In this chapter, we'll write React components' render methods in JSX, which is an HTML-like syntax. We'll also be using **hooks**, which were released in the v16.8 release — don't worry about versioning here, we'll get all set up in a minute. **Hooks** are a way to use state and lifecycle methods without making a class component, and also help share code between components. We'll even use our own custom hook!

But wait a minute, it seems like React and d3 are both used to create elements and update the DOM. To draw a chart, should we use both of libraries? Just one? Neither?

[82] https://reactjs.org/docs/hello-world.html

Having just gotten comfortable with d3, you probably aren't going to like the answer. Instead of using axis generators and letting d3 draw elements, **we're going to let React handle the rendering and to use d3 as a (very powerful) utility library**. Let's build an example chart library and see for ourselves why this makes the most sense.

Digging in

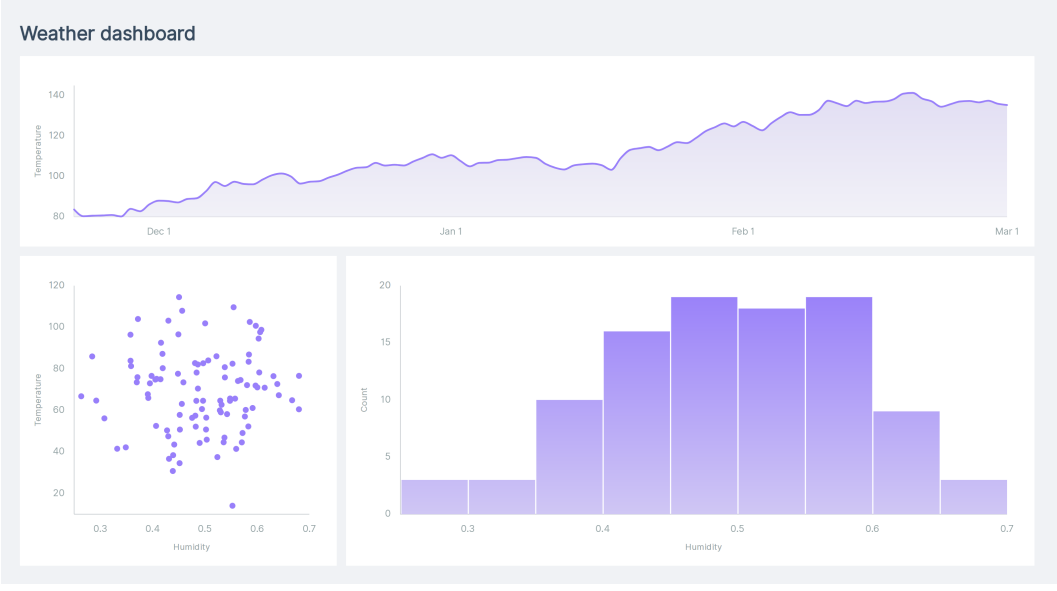
In the `/code/13-using-d3-with-react-js/` folder, you'll find a very bare bones React app. First, download the necessary packages with npm (or yarn, if you prefer).

```
npm install && npm install -D webpack-cli
```

Once your packages are installed, run

```
npm run start
```

and navigate to http://localhost:8090[83] in your browser. You should see an empty dashboard with three placeholders — one for a timeline, one for a scatter plot, and one for a histogram.



Finished dashboard

Within the `src` folder, we have an `App` component that is loading random data and updating it every four seconds — this will help us design our chart transitions.

Our chart-making plan has four levels of components:

1. **App**, which will decide what our dataset is and how to access values for our axes (accessors),
2. **Timeline**, **ScatterPlot**, or **Histogram** which will be re-useable components that decide how a specific type of chart is laid out and what goes in it,
3. **Chart**, which will pass down chart dimensions, and

[83] http://localhost:8090

4. **Axis**, **Line**, **Bars**, etc., which will create individual components within our charts.

Levels **3** and **4** will be isolated in the `Chart` folder, creating a charting library that can be used throughout the app to make many types of charts. Having a basic charting library will help in many ways, such as abstracting svg components idiosyncracies (for example, collaborators won't need to know that you need to use a `<rect>` to create a rectangle, and it takes a **x** attribute whereas a `<circle>` takes a **cx** attribute).

> If you're feeling lost or want to see the finished code, the completed charts and chart library are over in `src/completed` to help you out.

We'll start by fleshing out our `Timeline` component, running through our usual chart making steps.

1. **Access data**
2. **Create dimensions**
3. **Draw canvas**
4. **Create scales**
5. **Draw data**
6. **Draw peripherals**
7. **Set up interactions**

Access data

Let's open up our Timeline component, located in `src/Timeline.jsx`. There's not much in here: the bones of a React component, prop types, and all of the imports we'll need.

```
const Timeline = ({ data, xAccessor, yAccessor, label }) => {
  return (
    <div className="Timeline">
    </div>
  )
}
```

Our `Timeline` component takes four props:

1. `data`
2. `xAccessor`
3. `yAccessor`
4. `label`

These props are flexible enough to support throwing a timeline anywhere in our dashboard with any dataset. But we don't have so many props that creating a new timeline is overwhelming or allows us to create inconsistent timelines.

Create dimensions

Next up, we need to specify the size of our chart. In our dashboard, we could have Timelines of many different sizes. Each of these Timelines are also likely to change size based on the window size. To keep things flexible, we'll need to grab the dimensions of our container `<div>`.

We could implement this by hand by creating a React ref, querying the size of `ref.current`, and instantiating a Resize Observer to update on resize. Because we'll use this same logic in multiple chart types, we created a custom React hook called `useChartDimensions`.

`useChartDimensions` will accept an object of dimension overrides and return an array with two values:

1. a reference for a React ref
2. a `dimensions` object that looks like:

```
{
  width: 1000,
  height: 1000,
  marginTop: 100,
  marginRight: 100,
  marginBottom: 100,
  marginLeft: 100,
  boundedHeight: 800,
  boundedWidth: 800,
}
```

> To keep things simple, this object is flat, unlike some `dimensions` objects we've used before. In practice, if you need to rely on a specific structure for your `dimensions` object, it might be better to keep it flat instead of nesting `margins` inside another object.

We won't get into what that code looks like, but you can give it a look over in the `src/Chart/utils.js` file.

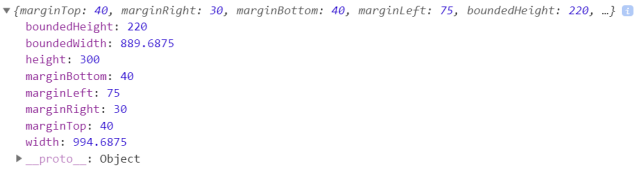
First, we'll use our hook and pull out the ref reference and the calculated dimensions.

```
const Timeline = ({ data, xAccessor, yAccessor, label }) => {
  const [ref, dimensions] = useChartDimensions()

  return (
    <div className="Timeline" ref={ref}>
    </div>
  )
}
```

Then, we will tag our container `<div>` with our `ref`.

```
<div className="Timeline" ref={ref}>
</div>
```

And voila! If we `console.log(dimensions)` before our render method, we can see that our `dimensions` are populated and update when we resize our window.



dimensions object

Draw canvas

Next up, we need to create our canvas. Since we'll want a canvas for all of our charts, we can put most of this logic in the `Chart` component. Let's add a `Chart` to our render method and pass it our `dimensions`.

```
<div className="Timeline" ref={ref}>
  <Chart dimensions={dimensions}>
  </Chart>
</div>
```

When we look at our dashboard again, not much has changed. Let's open up `src/Chart/Chart.jsx` to see what we're starting with.

`Chart` is a very basic functional React component — it asks for only one prop: `dimensions`.

> We're defining `useChartDimensions()` at the top of our file as an empty function to prevent import errors in another file. We'll update it in a minute.

```
export const useChartDimensions = () => {}

const Chart = ({ dimensions, children }) => (
  <svg className="Chart">
    { children }
  </svg>
)
```

The `children` in a `Chart` can be any component from our chart library (or raw SVG elements). Each of these components might need to know the dimensions of our chart — for example, an `Axis` component might need to know how tall to be. Instead of passing `dimensions` to each of these components as a prop, we can create a React Context that defines the dimensions for the whole chart.

First, we'll use the native React `createContext()` to create a new context — this code will go outside of the component, after our imports.

```
const ChartContext = createContext()
```

Next up, we can fill out our `useChartDimensions()` variable to create a more descriptive, easy-to-grab function that Chart components can use.

```
export const useChartDimensions = () => useContext(ChartContext)
```

Lastly, we need to add the context provider to our render method and specify that we want the context consumers to receive our `dimensions` object.

```
const Chart = ({ dimensions, children }) => (
  <ChartContext.Provider value={dimensions}>
    <svg className="Chart">
      { children }
    </svg>
  </ChartContext.Provider>
)
```

Great! Now all our chart components need to do to access the chart dimensions is to grab the value from `useChartDimensions()`.

Next up, we'll use those `dimensions` to create our chart **wrapper** and **bounds**. If you remember from **Chapter 1**, our **wrapper** spans the full height and width of the chart and the **bounds** respect the chart **margins**.

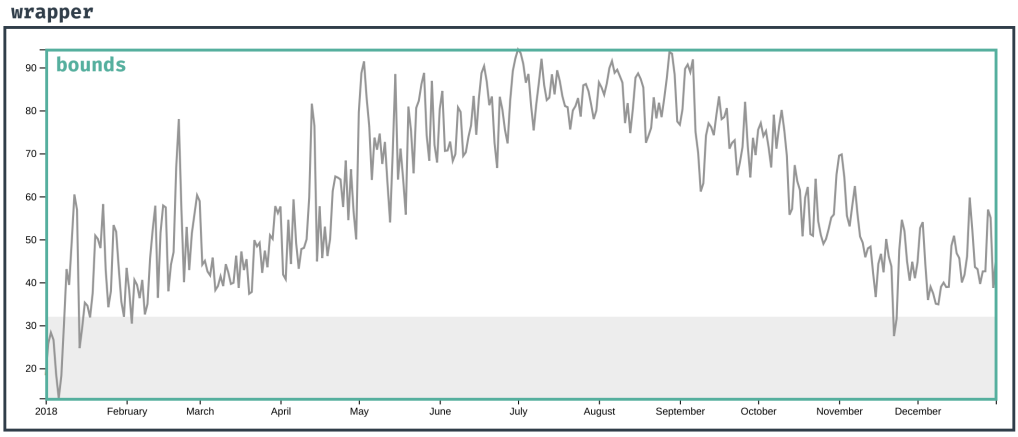


Chart dimensions

Let's specify the **width** and **height** of the `<svg>` element.

```
<svg
  className="Chart"
  width={dimensions.width}
  height={dimensions.height}>
  { children }
</svg>
```

Lastly, we'll create our chart bounds to shift our chart components and enforce our top and left margins.

```
<svg
  className="Chart"
  width={dimensions.width}
  height={dimensions.height}>
  <g transform={`translate(${
    dimensions.marginLeft
  }, ${
    dimensions.marginTop
  })`}>
    { children }
  </g>
</svg>
```

Perfect! Our `Chart` component is ready to go. We won't be able to see the difference on our webpage, but we can see our wrapper components in the **Elements** panel of our dev tools.

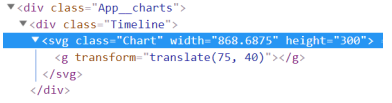


Chart elements

Create scales

Next up, we need to create the scales to convert from the data domain to the pixel domain. Let's pop back to `src/Timeline.jsx`.

We'll create scales just like we did in **Chapter 1** — we'll need an time-based **xScale** and a linear **yScale**.

```
const xScale = d3.scaleTime()
  .domain(d3.extent(data, xAccessor))
  .range([0, dimensions.boundedWidth])

const yScale = d3.scaleLinear()
  .domain(d3.extent(data, yAccessor))
  .range([dimensions.boundedHeight, 0])
  .nice()
```

> If you wanted to make creating scales easier, you could abstract the concept of a "scale" and add ease-of-use methods to your chart library. For example, you could make a method that takes a dimension (eg. `x`) and an accessor function and create a scale. A more comprehensive chart library can abstract redundant code and make it easier for collaborators who are less familiar with data visualization.

Next, we'll make a scaled accessor function for both of our axes. These will take a data point and return the pixel value. This way, our `Line` component won't need any knowledge of our scales or data structure.

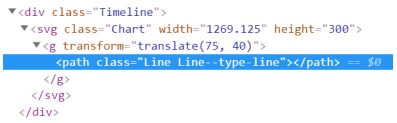
```
const xAccessorScaled = d => xScale(xAccessor(d))
const yAccessorScaled = d => yScale(yAccessor(d))
```

Draw data

We already imported our `Line` component from our chart library. Let's render one instance inside of our `Chart`, passing it our data and scaled accessor functions.

```
<Line
  data={data}
  xAccessor={xAccessorScaled}
  yAccessor={yAccessorScaled}
/>
```

If we inspect our webpage in the **Elements** panel, we can see a new `<path>` element.



Line element

Let's see what's going on in `src/Chart/Line.jsx`.

We have a basic element that renders a `<path>` element with a class name.

```
const Line = ({ type, data, xAccessor, yAccessor, y0Accessor, interpola\
tion, ...props }) => {
  return (
    <path {...props}
      className={`Line Line--type-${type}`}
    />
  )
}
```

`Line` accepts `data` and accessor props, along with a `type` string. A `Line` can have a `type` of `"line"` or `"area"` — it makes more sense to combine these two types of elements because they are more similar than they are different. There is one more prop (`interpolation`), which we'll get back to later.

Our first step is to create our `lineGenerator()`, which will turn our dataset into a `d` string for our `<path>`. Since `d3.line()` and `d3.area()` mimic our `type` prop, we can grab the right method with `d3[prop]()`.

```
const lineGenerator = d3[type]()
  .x(xAccessor)
  .y(yAccessor)
  .curve(interpolation)
```

`d3.area()` uses `.y0()` and `.y1()` to decide where the top and bottom of its path are. We'll need to add that extra logic only if we're creating an `area`.

```
if (type == "area") {
  lineGenerator
    .y0(y0Accessor)
    .y1(yAccessor)
}
```

Now we can use our `lineGenerator()` to convert our `data` into a `d` string.

```
<path {...props}
  className={`Line Line--type-${type}`}
  d={lineGenerator(data)}
/>
```

Nice! Now when we look at our webpage, we can see a squiggly line that updates every few seconds.

chart with line

Draw peripherals

Next, we want to draw our axes. This is where even experienced d3.js and React.js developers get confused because both libraries want to handle creating new the DOM elements. Up until now, we've used `d3.axisBottom()` and `d3.axisLeft()` to append multiple `<line>` and `<text>` elements to a manually created `<g>`. element. But the core concept of React.js relies on giving it full control over the DOM.

Let's first make a naive attempt at an **Axis** component, mimicking the d3.js code we've written so far. Since our **Axis** component is already imported, we can create a new instance in our render method. We'll need to specify the `dimension` and relevant `scale` of both of our axes.

```
<Axis
  dimension="x"
  scale={xScale}
/>
<Axis
  dimension="y"
  scale={yScale}
/>
```

> Remember that SVG elements' z-indices are determined by their order in the DOM. If you want your line to overlap your axes, make sure to add the`<Axis>` components before the `<Line>` in your render method.

Let's head over to `src/Chart/Axis-naive` to flesh out our **Axis** component. There's not much going on here yet, just a basic React Component that accepts a dimension (either **x** or **y**), a scale, and a tick formatting function.

```
const Axis = ({ dimension, scale, ...props }) => {
  return (
    <g {...props}
      className="Axis"
    />
  )
}
```

Let's start by pulling in the dimensions of our chart, using the custom React hook we created earlier.

```
const dimensions = useChartDimensions()
```

Since we'll want to use `d3.axisBottom()` or `d3.axisLeft()`, based on the `dimension` prop, let's make a map so we can dynamically grab the correct d3 method. This is one of many abstractions that can help to keep our chart library's API simple for collaborators less familiar with d3.

```
const axisGeneratorsByDimension = {
  x: "axisBottom",
  y: "axisLeft",
}
```

Now we can use our mapping to create a new axis generator, based on our `scale` prop.

```
const axisGenerator = d3[axisGeneratorsByDimension[dimension]]()
  .scale(scale)
```

In the past, we've used our axisGenerator on the d3 selection of a newly created `<g>` element. React gives us a way to access DOM nodes created in the render method: **Refs**. To create a React Ref, we create a new variable with `useRef()` and add it as a `ref` attribute to the element we want to target.

```
const ref = useRef()

return (
    <g {...props}
      ref={ref}
    />
)
```

Now when we access `ref.current`, we'll get a reference to the `<g>` DOM node.

Let's transform `ref.current` into a d3 selection by wrapping it with `d3.select()`, then transition our axis in using our `axisGenerator`.

> Note: we'll have to ensure that `ref.current` exists first, since this code will run before the first render.

```
if (ref.current) {
  d3.select(ref.current)
    .transition()
    .call(axisGenerator)
}
```

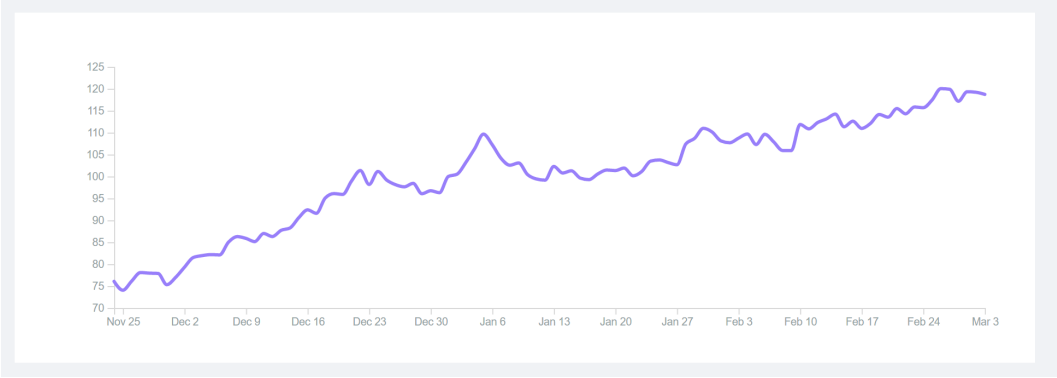
Just like that, we have axes! Something is missing though.

React timeline with axes

Right! We'll need to shift our x axis to the bottom of the chart. Let's add a `transform` attribute to our `<g>` element.

```
<g {...props}
  ref={ref}
  transform={
    dimension == "x"
      ? `translate(0, ${dimensions.boundedHeight})`
      : null
  }
/>
```

Perfect, now that looks like a timeline!



React timeline with axes, fixed

Axes, take two

Now that we've shown how easy it is to plug basic d3 code into React, let's talk about **why it's not a good idea**.

React uses a fancy reconciliation algorithm to decide which DOM elements to update. It does this by creating a virtual DOM and diffing against the real DOM. **When we mutate the DOM outside of React render methods, we're removing the performance benefits we get and will unnecessarily re-render elements**.

Short-cutting around React render methods also makes our code less declarative. Usually, you can depend on the resulting DOM to closely align to any JSX you see in a component's render method. When you come back to a component you wrote a few months ago, you'll thank yourself for making the output obvious.

In a pinch, using React to create a wrapper element to modify with d3 (like we just did) will do. You might need to do this for special cases, like animating an arc. But try to lean towards solely creating elements with React and using d3 as more of a utility library. This will keep your app speedy and less "hacky".

Even without its DOM modifying methods, d3 is a very powerful library. In fact, we created most of our timeline without needing to re-create a d3 generator function. For example, creating a `d` string for our **Line** component would have been tricky without `d3.line()`.

But how does this look in practice? Let's re-create our **Axis** component without using any axis generators.

Switch your Axis import in `src/Timeline.jsx` to use the `Axis` file.

```
import Axis from "./Chart/Axis"
```

When we open up `src/Chart/Axis.jsx`, we should see three basic React components.

Similar to how `d3.axisBottom()` and `d3.axisLeft()` are different methods, we want to split out `x` and `y` axes into different components. This will keep our code clean and prevent too many ternary statements.

The first component is a directory component that grabs the chart dimensions and renders the appropriate `Axis`, based on the `dimension` prop.

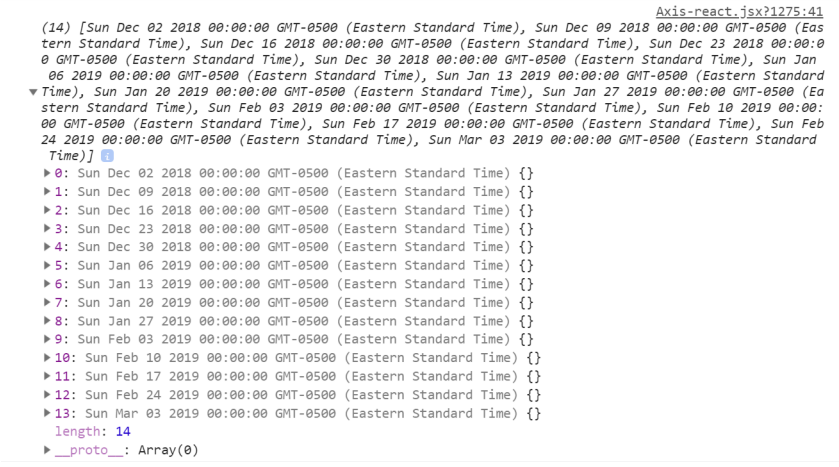
```
const axisComponentsByDimension = {
  x: AxisHorizontal,
  y: AxisVertical,
}
const Axis = ({ dimension, ...props }) => {
  const dimensions = useChartDimensions()
  const Component = axisComponentsByDimension[dimension]
  if (!Component) return null

  return (
    <Component {...props}
      dimensions={dimensions}
    />
  )
}
```

The other two components, `AxisHorizontal` and `AxisVertical`, are specific `Axis` implementations. Let's start by fleshing out `AxisHorizontal`.

```
function AxisHorizontal (
  { dimensions, label, formatTick, scale, ...props }
) {
  return (
    <g className="Axis AxisHorizontal" {...props}>
    </g>
  )
}
```

Since we're not using a d3 axis generator, we'll need to generate the ticks ourselves. Fortunately, many of the methods d3 uses internally are also available for external use. d3 scales have a `.ticks()` method that will create an array with evenly spaced values in the scale's domain.

We can see this in action if we `console.log(scale.ticks())`.



scale.ticks()

By default, `.ticks()` will aim for ten ticks, but we can pass a specific count to target. Note that `.ticks()` will *aim for* the count, but also tries to create ticks with meaningful intervals: a week in this example.

The number of ticks we want will depend on the chart width, though — ten ticks will likely crowd our x axis. Let's aim for one tick per 100 pixels for small screens and one tick per 250 pixels for wider screens.

```
function AxisHorizontal (
  { dimensions, label, formatTick, scale, ...props }
) {
  const numberOfTicks = dimensions.boundedWidth < 600
        ? dimensions.boundedWidth / 100
        : dimensions.boundedWidth / 250

  const ticks = scale.ticks(numberOfTicks)
```

Great! We're ready to render some elements. First, we'll shift our axis to the bottom of the chart. Remember: our `<Axis>` will render within our shifted group from `<Chart>`, so we don't have to worry about the top margin.

```
<g className="Axis AxisHorizontal" transform={`translate(0, ${
  dimensions.boundedHeight
})`} {...props}>
```

Most charts mark the end of the bounds with a line - let's draw a line above our axis to make it clear where the bottom of the y axis is.

Remember that `<line>` elements are positioned with `x1`, `x2`, `y1`, and `y2` attributes. We'll want to draw a line from `[0,0]` to `[width, 0]` — since `x1`, `x2`, and `y1` will all be `0` (the default), we can leave those attributes out.

```
<line
  className="Axis__line"
  x2={dimensions.boundedWidth}
/>
```

Next, we'll create the text for each of our ticks. To do this, we want to render a `<text>` element for each item in our `ticks` array. Let's shift each element down by 25px to give the axis line some breathing room and shift it to the right by converting the tick into the pixel domain using our `scale`.